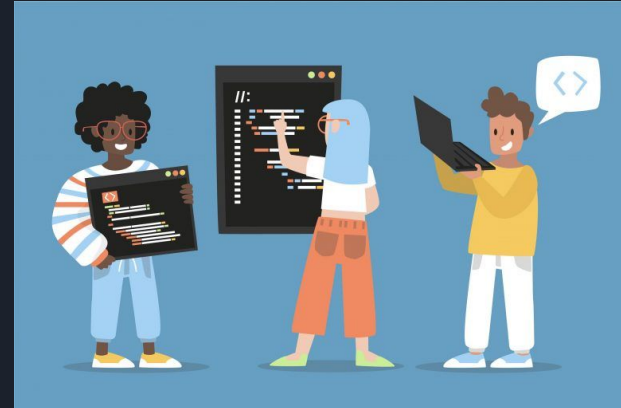


Evaluación en
Contacto con el
Docente

André Cevallos
25 julio del 2026

Introducción

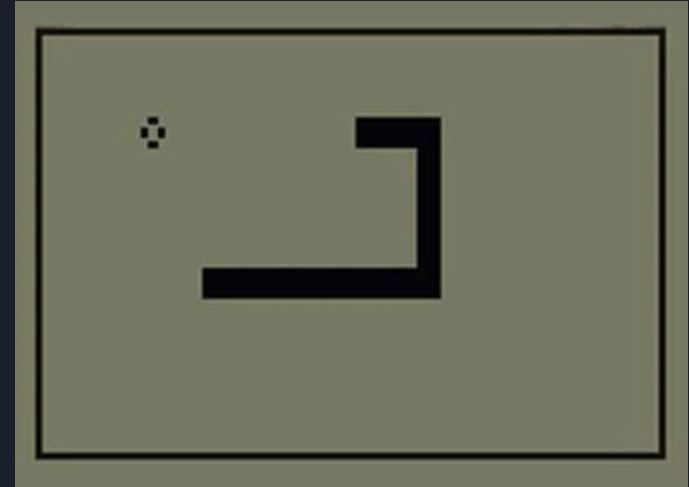
- Proyectos informáticos
 - Desarrollo del pensamiento lógico,
 - Resolución de problemas
 - Innovación
- Python y videojuegos
 - Motivación
 - Autoeficacia
 - Rendimiento académico de los estudiantes



Snake tech 🐍

Solución informática funcional

- análisis del problema
- diseño del sistema
- diagramas de funcionalidad
- diagramas de arquitectura de software
- estructuras lógicas de programación en Python
- control de versiones mediante GitHub





¿Desafío?



Objetivos

- Aplicar conocimientos de análisis, diseño y desarrollo de software
- Integrar los contenidos de la asignatura
- Utilizar GitHub para el control de versiones.
- Desarrollar un videojuego funcional
- Ampliar y consolidar los conocimientos adquiridos durante los aprendizajes autónomos 1 y 2.

Estructura del proyecto:

Diagrama de flujo

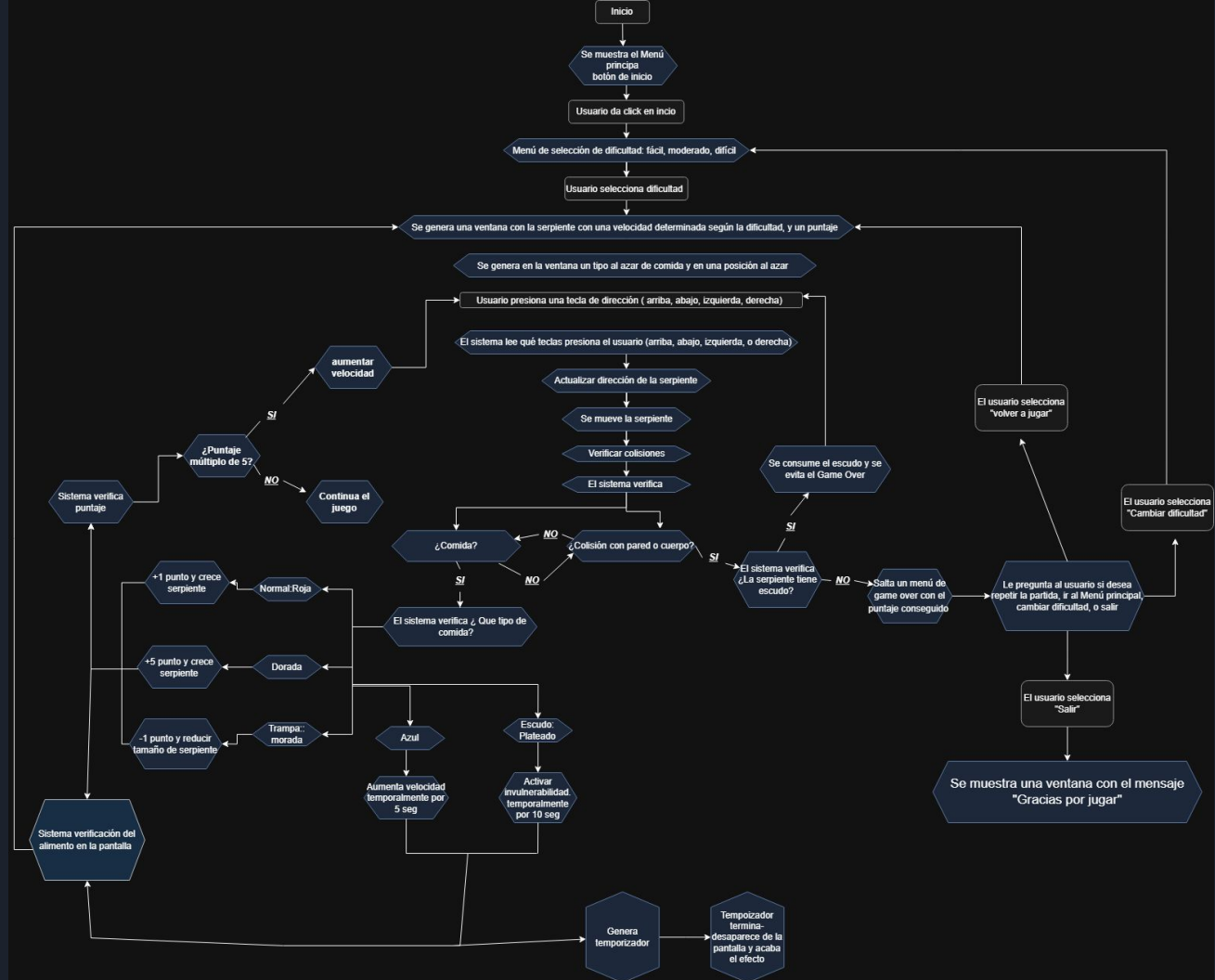


Diagrama de IPO

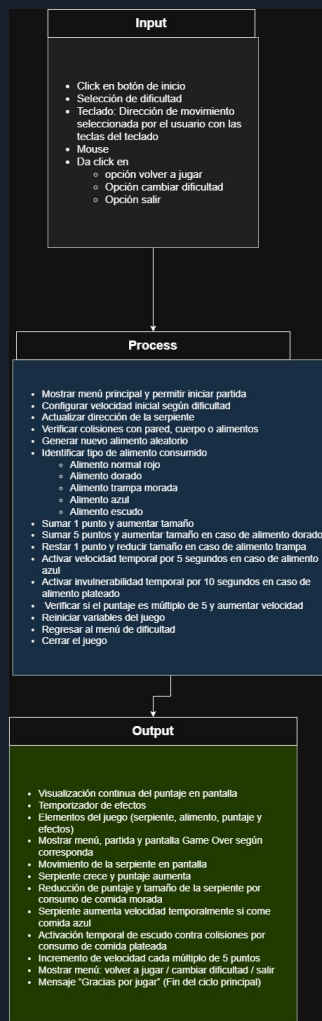


Diagrama de componentes

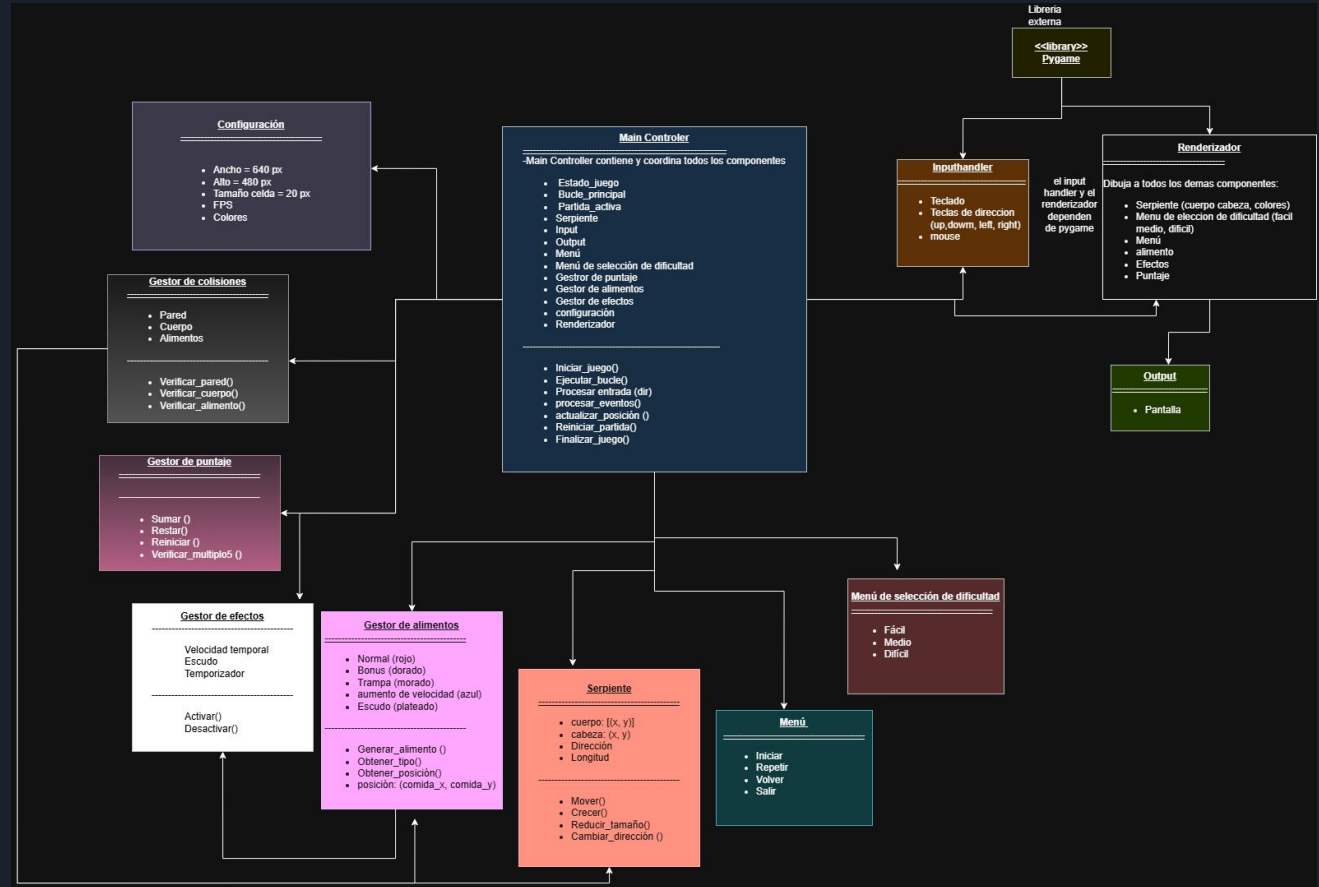


Diagrama de despliegue

Computadora/laptop del usuario

Sistema Operativo Windows

Entorno de ejecución

Python 3.12

Snake Tech

Main Controller que contiene y coordina todos los componentes

- Estado_juego
- Bucle_principal
- Partida_activa
- Serpiente
- Input
- Output
- Menú
- Menú de selección de dificultad
- Gestor de puntaje
- Gestor de alimentos
- Gestor de efectos
- configuración
- Renderizador

<<library>> Pygame. Biblioteca externa para el desarrollo de videojuegos en Python. Gestiona la ventana gráfica, entrada del usuario, renderizado y actualización de elementos en pantalla.

Pantalla
Salida visual del videojuego

Periféricos:
Teclado y mouse.

Estructura general del código

Ciclo principal (while True) : Juego en ejecución

Variable **estado**:

- Organiza
- facilita
- propia lógica

```
direccion = 0

velocidad_base = 5
velocidad = velocidad_base

turbo_activo = False
tiempo_turbo = 0

escudo_activo = False
tiempo_escudo = 0

estado = "menu"

while True:

    if estado == "menu":
        ventana.fill((245, 245, 245))

        titulo = fuente.render("SNAKE TECH", True, (0,120,0))
        titulo_rect = titulo.get_rect(center=(ANCHO//2, 90))
        ventana.blit(titulo, titulo_rect)

        boton_inicio = pygame.Rect(220, 100, 200, 55)
```

```
462
463     ventana.blit(texto_escudo, (10, 70))
464
465     pygame.display.update()
466     reloj.tick(velocidad)
467
```



Configuración inicial del juego

```
ANCHO = 640
```

```
ALTO = 480
```

```
ventana = pygame.display.set_mode((ANCHO, ALTO))
```

```
pygame.display.set_caption("Snake Tech")
```

```
fuentes = pygame.font.SysFont("Arial", 25)
```

```
reloj = pygame.time.Clock()
```



Importación de bibliotecas

Importación:

```
import pygame  
import random  
  
pygame.init()
```



Variables

Python

```
x = 300
```

```
y = 200
```

```
tamano = 20
```

```
puntos = 0
```

```
velocidad_base = 5
```

```
velocidad = velocidad_base
```

```
serpiente = []
```

```
longitud_serpiente = 1
```

```
comida_x = 100
```

```
comida_y = 100
```

```
tipo_comida = "normal"
```

Estados del programa

```
estado = "menu"
```

```
while True:
```

```
    if estado == "menu":
```

```
        ventana.fill((245, 245, 245))
```



Condicionales
if

len(serpiente)

max()

int()

serpiente.pop(0)

```
if tipo_comida == "dorada":
    puntos = puntos + 5
    longitud_serpiente = longitud_serpiente + 1

if tipo_comida == "trampa":
    puntos = puntos - 1
    if longitud_serpiente > 1:
        longitud_serpiente = max(1, int(longitud_serpiente * 0.5))
    while len(serpiente) > longitud_serpiente:
        serpiente.pop(0)
```

Operadores
racionales: <, >, ==

Operadores lógicos:
or, not, and

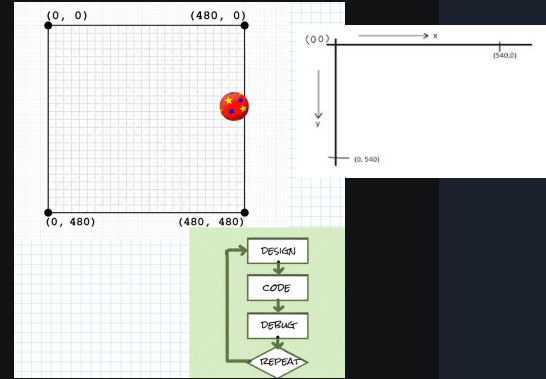
operador modulo

```
if x < 0 or x > ANCHO - tamaño or y < 0 or y > ALTO - tamaño:
```

```
    if escudo_activo:  
        escudo_activo = False
```

```
    if x < 0:  
        x = 0  
    if x > ANCHO - tamaño:  
        x = ANCHO - tamaño  
    if y < 0:  
        y = 0  
    if y > ALTO - tamaño:  
        y = ALTO - tamaño
```

```
else:  
    estado = "game_over"  
    continue
```



```
if puntos > 0 and puntos % 5 == 0:  
    velocidad_base = velocidad_base + 1
```

```
if not turbo_activo:  
    velocidad = velocidad_base
```



Bucles: Función While

```
while True:  
    if estado == "menu":  
        ventana.fill((245, 245, 245))
```




Bucles:
Función For

Eventos

```
for evento in pygame.event.get():  
    if evento.type == pygame.QUIT:  
        pygame.quit()  
        exit()  
  
    if evento.type == pygame.KEYDOWN:  
        if evento.key == pygame.K_RIGHT:  
            direccion_x = tamano  
            direccion_y = 0  
  
        if evento.key == pygame.K_LEFT:  
            direccion_x = -tamano  
            direccion_y = 0  
  
        if evento.key == pygame.K_UP:  
            direccion_x = 0  
            direccion_y = -tamano  
  
        if evento.key == pygame.K_DOWN:  
            direccion_x = 0  
            direccion_y = tamano
```



Lista

```
serpiente = []  
longitud_serpiente = 1
```

Tuplas →
append()

```
cabeza = (x, y)  
serpiente.append(cabeza)
```

```
if len(serpiente) > longitud_serpiente:  
    serpiente.pop(0)
```

○ →

```
color_iniciar = (0, 220, 0)  
color_salir = (220, 0, 0)  
color_comida = (255, 215, 0)  
ventana.fill((245, 245, 245))
```



Random

```
print("Comiste la comida")
print("Puntos:", puntos)

comida_x = random.randrange(0, ANCHO, tamano)
comida_y = random.randrange(0, ALTO, tamano)

tipo_comida = random.choice(["normal", "normal", "normal", "dorada", "trampa", "azul", "escudo"])
```

pygame.time.get_ticks()

```
if tipo_comida == "azul":  
    puntos = puntos + 1  
    longitud_serpiente = longitud_serpiente + 1  
  
    turbo_activo = True  
    tiempo_turbo = pygame.time.get_ticks()  
    velocidad = velocidad_base + 5  
  
    if tiempo_actual - tiempo_turbo > 5000:  
        turbo_activo = False  
        velocidad = velocidad_base
```



pygame.draw.rect()

render()

str()

ventana.blit()

```
pygame.draw.rect(ventana, color_comida,  
                 (comida_x, comida_y, tamano, tamano))
```

```
texto_puntos = fuente.render("Puntos: " + str(puntos), True, (0, 0, 0))  
ventana.blit(texto_puntos, (10, 10))
```



ventana.fill()

get_rect() center=()

display.update()

pygame.time.delay()

pygame.quit()

```
if estado == "salir":
```

```
    ventana.fill((245, 245, 245))
```

```
    texto1 = fuente.render("¡Gracias por jugar!", True, (0, 120, 0))
```

```
    texto2 = fuente.render("Snake Tech", True, (0, 0, 0))
```

```
    rect1 = texto1.get_rect(center=(ANCHO//2, 200))
```

```
    rect2 = texto2.get_rect(center=(ANCHO//2, 245))
```

```
    ventana.blit(texto1, rect1)
```

```
    ventana.blit(texto2, rect2)
```

```
pygame.display.update()
```

```
pygame.time.delay(2500)
```

```
pygame.quit()
```

```
exit()
```



```
pygame.font.SysFont()
```

```
pygame.time.Clock()
```

```
reloj.tick()
```

```
f fuente = pygame.font.SysFont("Arial", 25)
reloj = pygame.time.Clock()
```

```
pygame.display.update()
reloj.tick(velocidad)
```



pygame.mouse.get_pos()

collidepoint(mouse)

```
mouse = pygame.mouse.get_pos()

if boton_iniciar.collidepoint(mouse):
|   color_iniciar = (0, 220, 0)
else:
|   color_iniciar = (0, 180, 0)

if boton_salir.collidepoint(mouse):
|   color_salir = (220, 0, 0)
else:
|   color_salir = (180, 0, 0)
```




Fin

